CODES DES DIAGRAMMES UML – TP 3

Système de gestion d'une résidence universitaire • POO en C++

Ce document contient les quatre codes sources au format PlantUML. Pour générer un diagramme, copie un bloc complet depuis `@startuml` jusqu'à `@enduml` dans un éditeur PlantUML.

1. Diagramme de cas d'utilisation

```
@startuml
left to right direction
actor "Responsable de résidence" as R
rectangle "Système de gestion de résidence" {
 usecase "Enregistrer une demande\nd'hébergement" as U1
 usecase "Attribuer une chambre" as U2
 usecase "Vérifier les places\ndisponibles" as U3
 usecase "Enregistrer un départ" as U4
 usecase "Consulter les chambres\nlibres et occupées" as U5
 usecase "Consulter l'historique\ndes occupations" as U6
 usecase "Afficher les statistiques" as U7
}
R --> U1
R --> U2
R --> U4
R --> U5
R --> U6
R --> U7
U2 .> U3 : <<include>>
@enduml
```

2. Diagramme de classes

```
@startuml
skinparam classAttributeIconSize 0

class Etudiant {
 - id : int
 - nom : string
 - prenom : string
 - filiere : string
 + afficher() : void
}
class Residence {
 - nom : string
 - adresse : string
 + ajouterBatiment(b : Batiment) : void
 + tauxOccupation() : double
 + afficherStatistiques() : void
}
class Batiment {
 - code : string
 - nom : string
 + ajouterChambre(c : Chambre) : void
}
class Chambre {
 - numero : string
 - capacite : int
 - litsOccupes : int
 + placesDisponibles() : int
 + estDisponible() : bool
 + occuperUnePlace() : bool
 + libererUnePlace() : void
}
class DemandeHebergement {
 - id : int
 - dateDemande : Date
 - statut : StatutDemande
 + accepter() : void
 + refuser() : void
 + annuler() : void
}
class Attribution {
 - id : int
 - dateDebut : Date
 - dateFin : Date
 - active : bool
 + cloturer(date : Date) : void
 + estActive() : bool
}
enum StatutDemande {
 EN_ATTENTE
 EN_ETUDE
 ACCEPTEE
 REFUSEE
 SATISFAITE
 ANNULEE
}
Residence "1" *-- "1..*" Batiment
Batiment "1" *-- "1..*" Chambre
Etudiant "1" -- "0..*" DemandeHebergement
Etudiant "1" -- "0..*" Attribution
Chambre "1" -- "0..*" Attribution
DemandeHebergement "1" -- "0..1" Attribution
DemandeHebergement --> StatutDemande
@enduml
```

3. Diagramme de séquence – Attribution

```
@startuml
actor Responsable
participant "GestionResidence" as Gestion
participant "DemandeHebergement" as Demande
participant "Chambre" as Chambre
participant "Attribution" as Attribution

Responsable -> Gestion : attribuer(demande, chambre)
Gestion -> Demande : verifierStatut()
Demande --> Gestion : statut valide ?
Gestion -> Chambre : placesDisponibles()
Chambre --> Gestion : nombre de places

alt chambre disponible et demande valide
 Gestion -> Chambre : occuperUnePlace()
 Chambre --> Gestion : true
 create Attribution
 Gestion -> Attribution : new(etudiant, chambre, dateDebut)
 Gestion -> Demande : setStatut(SATISFAITE)
 Gestion --> Responsable : confirmation
else chambre complète ou demande invalide
 Gestion --> Responsable : échec + motif
end
@enduml
```

4. Diagramme d'états-transitions – Demande

```
@startuml
[*] --> EnAttente : création de la demande
EnAttente --> EnEtude : examen du dossier
EnEtude --> Acceptee : décision favorable
EnEtude --> Refusee : décision défavorable
Acceptee --> Satisfaite : chambre attribuée
EnAttente --> Annulee : annulation
EnEtude --> Annulee : annulation
Acceptee --> Annulee : annulation avant attribution
Satisfaite --> [*]
Refusee --> [*]
Annulee --> [*]
@enduml
```

Remarque : les noms, attributs, statuts et relations suivent le corrigé proposé. Ils peuvent être ajustés aux consignes précises de ton enseignant.